

###1. Problem Definition

Objective:

To determine whether a Personal Loan should be approved and recommend repayment terms using customer financial & credit attributes.

Target Variable: loan_status

- Fully Paid $\rightarrow 1$
- Charged Off $\rightarrow 0$

This is a binary classification problem.

###2. Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import roc_curve, roc_auc_score
from sklearn.metrics import precision_recall_curve, auc
from scipy.stats import spearmanr
```

###3. Load Dataset

```
df = pd.read_csv("LoanTap.csv")  
print("Shape:", df.shape)  
df.head()  
  
Shape: (396030, 27)  
  
{"type": "dataframe", "variable name": "df"}
```

#4. Structure & Summary

```
df.info()
df.describe()
```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 396030 entries, 0 to 396029
Data columns (total 27 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	----

0	loan_amnt	396030	non-null	float64
1	term	396030	non-null	object
2	int_rate	396030	non-null	float64
3	installment	396030	non-null	float64
4	grade	396030	non-null	object
5	sub_grade	396030	non-null	object
6	emp_title	373103	non-null	object
7	emp_length	377729	non-null	object
8	home_ownership	396030	non-null	object
9	annual_inc	396030	non-null	float64
10	verification_status	396030	non-null	object
11	issue_d	396030	non-null	object
12	loan_status	396030	non-null	object
13	purpose	396030	non-null	object
14	title	394274	non-null	object
15	dti	396030	non-null	float64
16	earliest_cr_line	396030	non-null	object
17	open_acc	396030	non-null	float64
18	pub_rec	396030	non-null	float64
19	revol_bal	396030	non-null	float64
20	revol_util	395754	non-null	float64
21	total_acc	396030	non-null	float64
22	initial_list_status	396030	non-null	object
23	application_type	396030	non-null	object
24	mort_acc	358235	non-null	float64
25	pub_rec_bankruptcies	395495	non-null	float64
26	address	396030	non-null	object

```
dtypes: float64(12), object(15)
```

```
memory usage: 81.6+ MB
```

```
{
  "summary": {
    "name": "df",
    "rows": 8,
    "fields": [
      {
        "column": "loan_amnt",
        "properties": {
          "dtype": "number",
          "std": 135325.93918604977,
          "min": 500.0,
          "max": 396030.0,
          "num_unique_values": 8,
          "samples": [
            14113.888089286165,
            12000.0,
            396030.0
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "int_rate",
        "properties": {
          "dtype": "number",
          "std": 140012.96485144028,
          "min": 4.472157381532854,
          "max": 396030.0,
          "num_unique_values": 8,
          "samples": [
            13.6394000454511,
            13.33,
            396030.0
          ],
          "semantic_type": "",
          "description": ""
        },
        "column": "installment",
        "properties": {
          "dtype": "number",
          "std": 139845.47529700142,
          "min": 396030.0,
          "max": 396030.0,
          "num_unique_values": 8,
          "samples": [
            431.84969802792716,
            375.43,
            396030.0
          ],
          "semantic_type": ""
        }
      ]
    }
  }
}
```

```

\"description\": \"\"\\n      }\\n    },\\n    {\\n      \"column\":
\"annual_inc\",\\n      \"properties\": {\\n        \"dtype\":
\"number\",\\n        \"std\": 3043769.9082227983,\\n        \"min\":
0.0,\\n        \"max\": 8706582.0,\\n        \"num_unique_values\": 8,\\n
\"samples\": [\\n          74203.17579771738,\\n          64000.0,\\n
396030.0\\n        ],\\n        \"semantic_type\": \"\",\\n
\"description\": \"\"\\n      }\\n    },\\n    {\\n      \"column\":
\"dti\",\\n      \"properties\": {\\n        \"dtype\": \"number\",\\n
\"std\": 139552.09502751552,\\n        \"min\": 0.0,\\n        \"max\":
396030.0,\\n        \"num_unique_values\": 8,\\n        \"samples\": [\\n
17.379513647955967,\\n        16.91,\\n        396030.0\\n      ],\\n
      \"semantic_type\": \"\",\\n      \"description\": \"\"\\n
    }\\n  },\\n  {\\n    \"column\": \"open_acc\",\\n
\"properties\": {\\n      \"dtype\": \"number\",\\n      \"std\":
140010.75955392266,\\n      \"min\": 0.0,\\n      \"max\":
396030.0,\\n      \"num_unique_values\": 8,\\n      \"samples\": [\\n
11.31115319546499,\\n      10.0,\\n      396030.0\\n    ],\\n
    \"semantic_type\": \"\",\\n    \"description\": \"\"\\n
  }\\n  },\\n  {\\n    \"column\": \"pub_rec\",\\n    \"properties\":
{\\n      \"dtype\": \"number\",\\n      \"std\":
140013.37304312378,\\n      \"min\": 0.0,\\n      \"max\":
396030.0,\\n      \"num_unique_values\": 5,\\n      \"samples\": [\\n
0.17819104613286874,\\n      86.0,\\n      0.530670600472675\\n
    ],\\n    \"semantic_type\": \"\",\\n    \"description\": \"\"\\n
  }\\n  },\\n  {\\n    \"column\": \"revol_bal\",\\n
\"properties\": {\\n      \"dtype\": \"number\",\\n      \"std\":
607701.8324003755,\\n      \"min\": 0.0,\\n      \"max\":
1743266.0,\\n      \"num_unique_values\": 8,\\n      \"samples\": [\\n
15844.539853041437,\\n      11181.0,\\n      396030.0\\n    ],\\n
    \"semantic_type\": \"\",\\n    \"description\": \"\"\\n
  }\\n  },\\n  {\\n    \"column\": \"revol_util\",\\n
\"properties\": {\\n      \"dtype\":
\"number\",\\n      \"std\": 139863.2101518472,\\n      \"min\":
0.0,\\n      \"max\": 395754.0,\\n      \"num_unique_values\": 8,\\n
\"samples\": [\\n        53.79174863677942,\\n        54.8,\\n
395754.0\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\"\\n    }\\n  },\\n  {\\n    \"column\":
\"total_acc\",\\n    \"properties\": {\\n      \"dtype\":
\"number\",\\n      \"std\": 140004.4584781143,\\n      \"min\":
2.0,\\n      \"max\": 396030.0,\\n      \"num_unique_values\": 8,\\n
\"samples\": [\\n        25.41474383253794,\\n        24.0,\\n
396030.0\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\"\\n    }\\n  },\\n  {\\n    \"column\":
\"mort_acc\",\\n    \"properties\": {\\n      \"dtype\":
\"number\",\\n      \"std\": 126653.08000228765,\\n      \"min\":
0.0,\\n      \"max\": 358235.0,\\n      \"num_unique_values\": 7,\\n
\"samples\": [\\n        358235.0,\\n        1.8139908160844138,\\n
3.0\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\"\\n    }\\n  },\\n  {\\n    \"column\":

```

```
\ "pub_rec_bankruptcies\ ",\n      \ "properties\ ": {\n          \ "dtype\ ":
\ "number\ ",\n          \ "std\ ": 139828.1700455425,\n          \ "min\ ":
0.0,\n          \ "max\ ": 395495.0,\n          \ "num_unique_values\ ": 5,\n
\ "samples\ ": [\n          0.12164755559488742,\n          8.0,\n
0.3561742766277696\n          ],\n          \ "semantic_type\ ": \ "\",\n
\ "description\ ": \ "\",\n          }\n      }\n  ]\n }", "type": "dataframe"}
```

#Insight:

Large dataset (~396K rows)

Mix of categorical and numeric features

Good sample size for stable underwriting

###5. Duplicate Check

```
print("Duplicate rows:", df.duplicated().sum())
df.drop_duplicates(inplace=True)
```

Duplicate rows: 0

#Insight:

Removing duplicates prevents biased training.

#6. Target Mapping

```
df['loan_status'] = df['loan_status'].str.lower()
df = df[df['loan_status'].isin(['fully paid', 'charged off'])]
df['target'] = df['loan_status'].map({
    'fully paid': 1,
    'charged off': 0
})
df['target'].value_counts(normalize=True)

target
1    0.803871
0    0.196129
Name: proportion, dtype: float64

df.drop(columns=['loan_status'], inplace=True)
```

#Insight:

Around 80% customers fully paid, ~20% defaulted. Dataset is imbalanced.

#7. Missing Value Detection

```
df.isnull().sum().sort_values(ascending=False).head(10)
```

```
mort_acc          37795
emp_title         22927
emp_length        18301
title             1756
pub_rec_bankruptcies  535
revol_util         276
installment        0
int_rate           0
term               0
grade              0
dtype: int64
```

```
df.fillna(df.median(numeric_only=True), inplace=True)
```

#Insight:

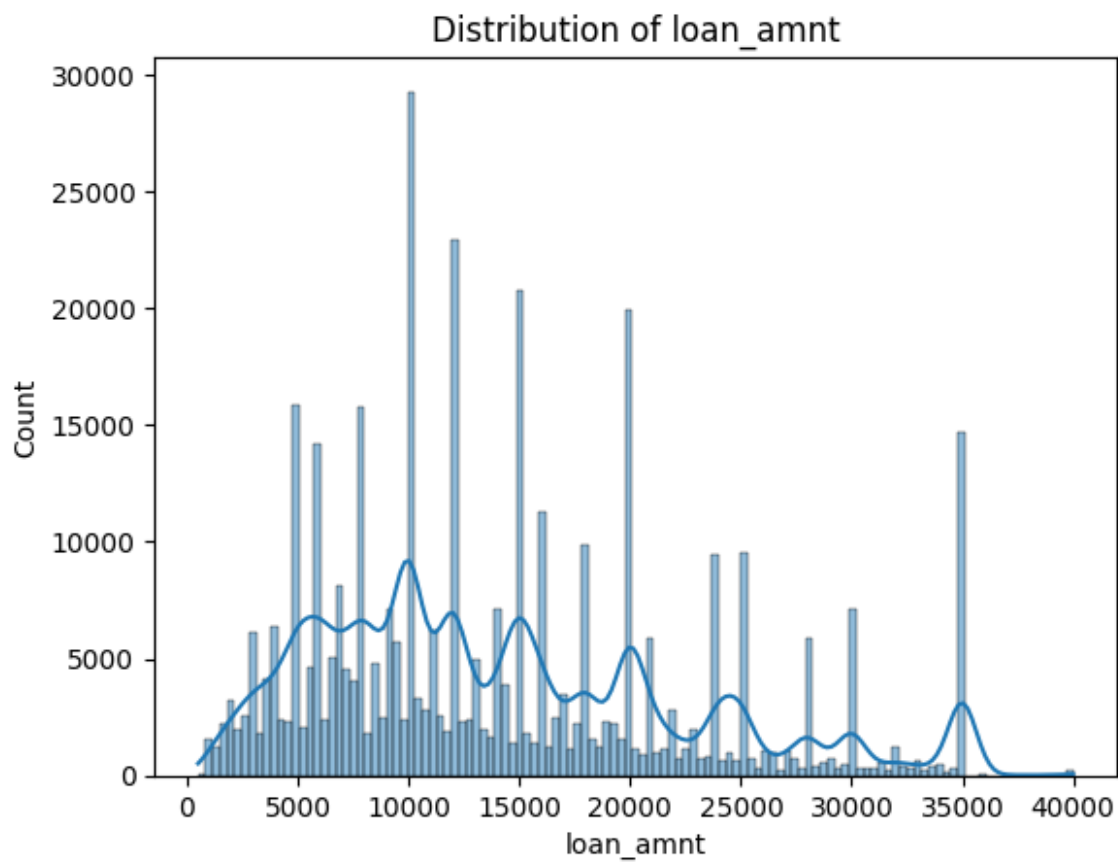
Median imputation prevents skew distortion.

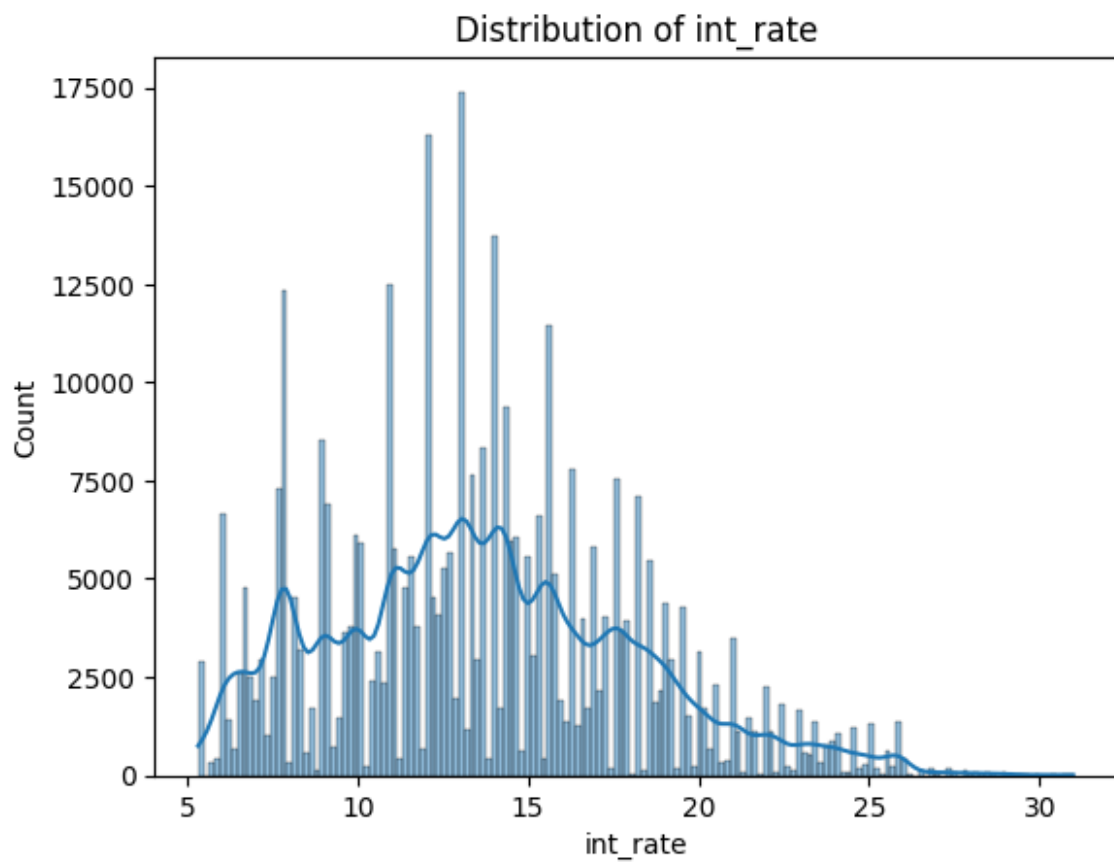
#8. Univariate Analysis

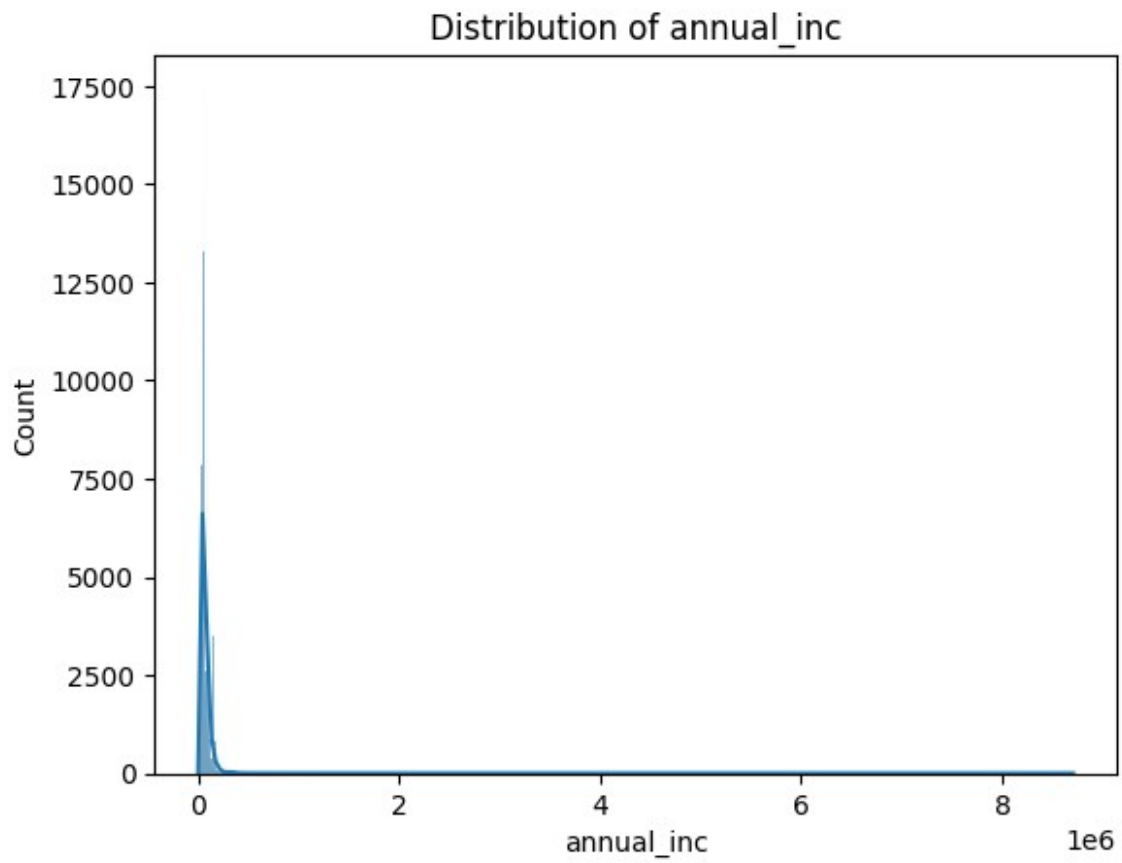
#Continuous Distribution

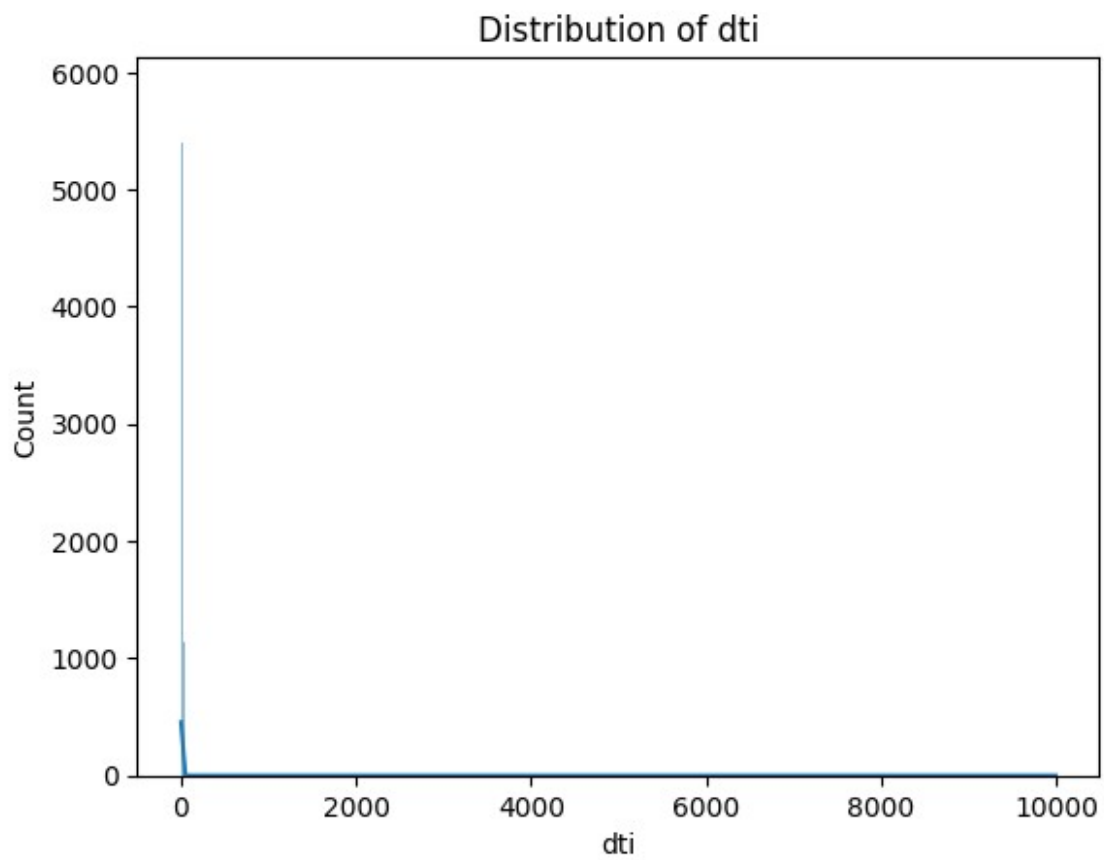
```
continuous_cols =
['loan_amnt', 'int_rate', 'annual_inc', 'dti', 'revol_util']

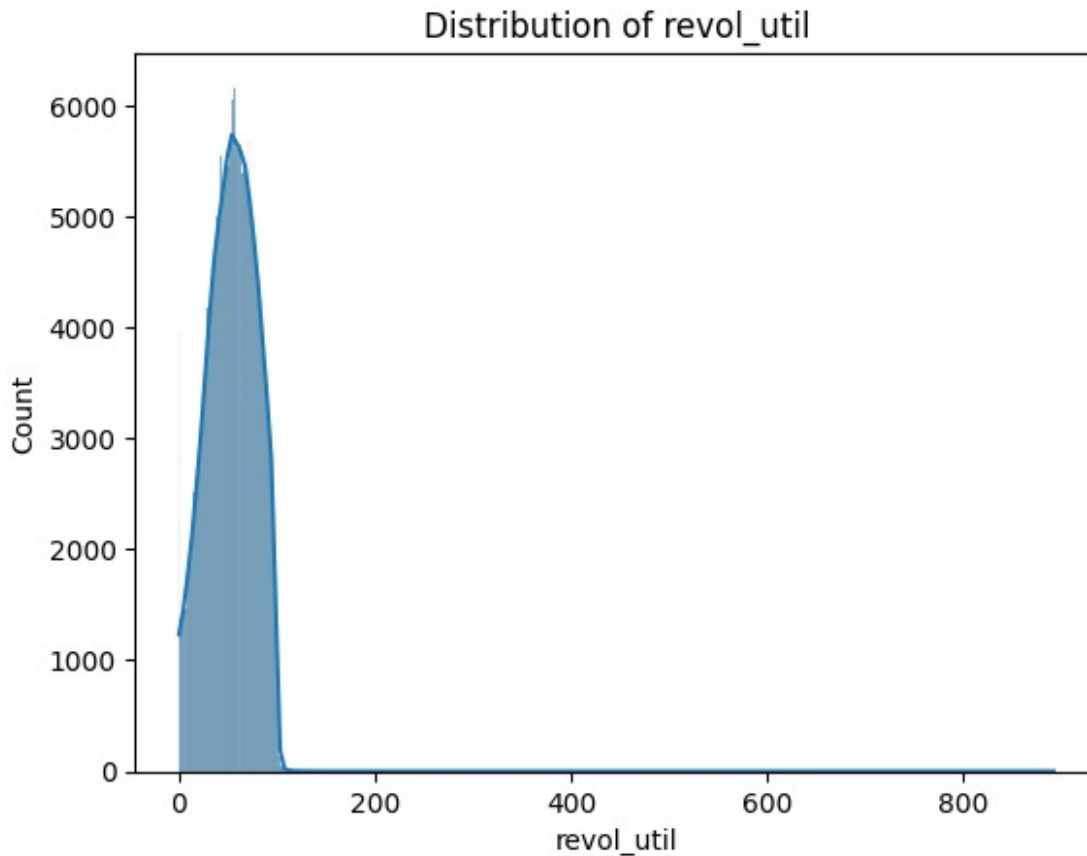
for col in continuous_cols:
    plt.figure()
    sns.histplot(df[col], kde=True)
    plt.title(f'Distribution of {col}')
    plt.show()
```











#9. Spearman Correlation (Multicollinearity)

```
df[['loan_amnt', 'installment']].corr(method='spearman')

{"summary": "{\n  \"name\": \"df[['loan_amnt', 'installment']]\", \n  \"rows\": 2, \n  \"fields\": [\n    {\n      \"column\": \"loan_amnt\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.022391449948593493, \n        \"min\": 0.9683337078015007, \n        \"max\": 1.0, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          0.9683337078015007, \n          1.0\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }, \n    {\n      \"column\": \"installment\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 0.022391449948593493, \n        \"min\": 0.9683337078015007, \n        \"max\": 1.0, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          1.0, \n          0.9683337078015007\n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\"\n      }\n    }\n  ]\n}", "type": "dataframe"}
```

#Business Insight:

Spearman correlation $\approx 0.97 \rightarrow$ High multicollinearity $\rightarrow$ Drop one variable (installment derived from loan_amnt)

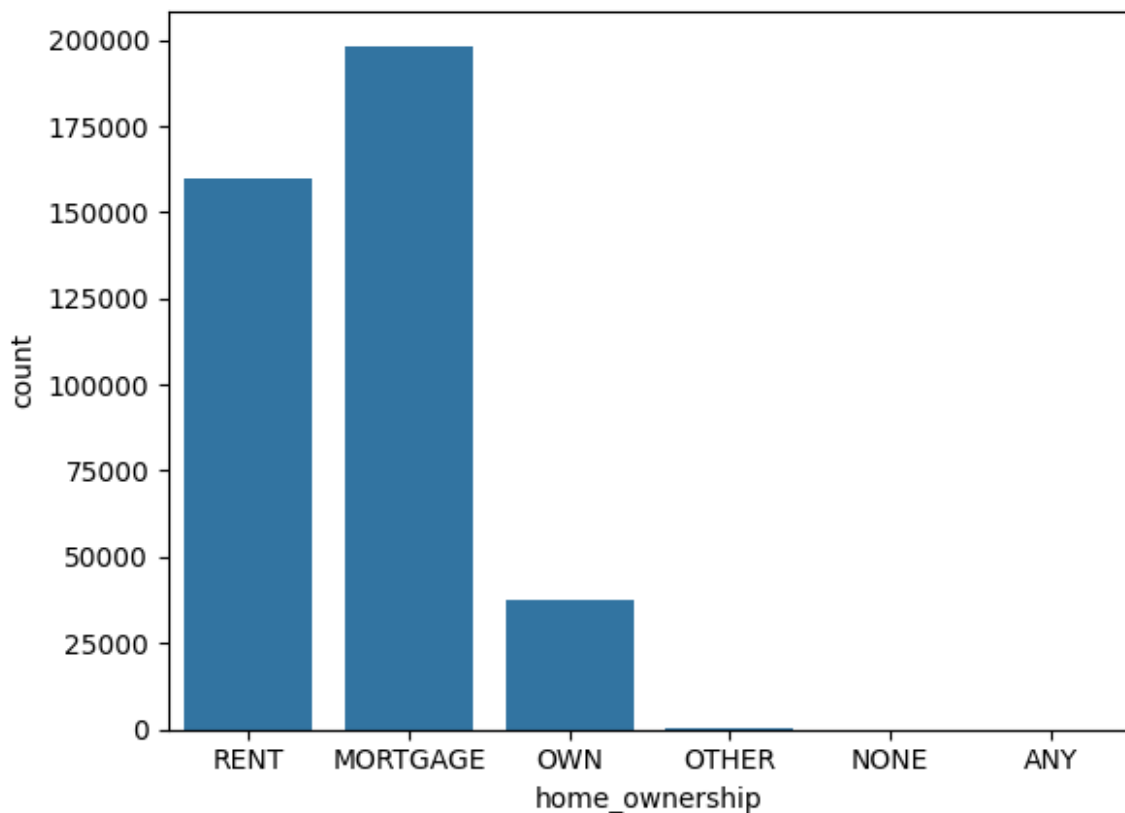
```
df.drop(columns=['installment'], inplace=True)
```

#10. Home Ownership

```
df['home_ownership'].value_counts(normalize=True)
```

```
home_ownership
MORTGAGE    0.500841
RENT        0.403480
OWN         0.095311
OTHER       0.000283
NONE        0.000078
ANY         0.000008
Name: proportion, dtype: float64
```

```
sns.countplot(x='home_ownership', data=df)
plt.show()
```



#Majority: MORTGAGE (~50%)

#10. Grade vs Repayment

```
pd.crosstab(df['grade'], df['target'], normalize='index')
```

```
{
  "summary": {
    "name": "pd",
    "rows": 7,
    "fields": [
      {
        "column": "grade",
        "properties": {
          "dtype": "string",
          "num_unique_values": 7,
          "samples": [
            "A",
            "B",
            "F"
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": 0,
        "properties": {
          "dtype": "number",
          "std": 0.15580354529196594,
          "min": 0.0628787760761525,
          "max": 0.47838899803536344,
          "num_unique_values": 7,
          "samples": [
            0.0628787760761525,
            0.12573049009636436,
            0.4278797145769623
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": 1,
        "properties": {
          "dtype": "number",
          "std": 0.15580354529196597,
          "min": 0.5216110019646365,
          "max": 0.9371212239238476,
          "num_unique_values": 7,
          "samples": [
            0.9371212239238476,
            0.8742695099036356,
            0.5721202854230377
          ],
          "semantic_type": "",
          "description": ""
        }
      }
    ]
  },
  "type": "dataframe"
}
```

Insight:

Grade A → ~93% repayment

TRUE: People with Grade A more likely to repay.

#12. Top 2 Job Titles

```
df['emp_title'].value_counts().head(2)
```

```
emp_title
Teacher      4389
Manager      4250
Name: count, dtype: int64
```

#13. Date Feature Engineering

#Converting month columns into numeric credit history length.

```
df['issue_d'] = pd.to_datetime(df['issue_d'], errors='coerce')
df['earliest_cr_line'] = pd.to_datetime(df['earliest_cr_line'],
errors='coerce')

df['credit_history_months'] = (
    (df['issue_d'].dt.year - df['earliest_cr_line'].dt.year) * 12 +
    (df['issue_d'].dt.month - df['earliest_cr_line'].dt.month)
)
```

/tmp/ipython-input-3609922344.py:1: UserWarning: Could not infer format, so each element will be parsed individually, falling back to

```

`dateutil`. To ensure parsing is consistent and as-expected, please
specify a format.
df['issue_d'] = pd.to_datetime(df['issue_d'], errors='coerce')
/tmp/ipython-input-3609922344.py:2: UserWarning: Could not infer
format, so each element will be parsed individually, falling back to
`dateutil`. To ensure parsing is consistent and as-expected, please
specify a format.
df['earliest_cr_line'] = pd.to_datetime(df['earliest_cr_line'],
errors='coerce')

df[['issue_d', 'earliest_cr_line', 'credit_history_months']].head()

{"summary": "{\n  \"name\":\n  \"df[['issue_d', 'earliest_cr_line', 'credit_history_months']]\", \n
  \"rows\": 5, \n  \"fields\": [\n    {\n      \"column\": \"issue_d\", \n
      \"properties\": {\n        \"dtype\": \"date\", \n        \"min\":\n        \"2013-04-01 00:00:00\", \n        \"max\": \"2015-01-01 00:00:00\", \n
        \"num_unique_values\": 3, \n        \"samples\": [\n          \"2015-01-01 00:00:00\", \n          \"2014-11-01 00:00:00\", \n          \"2013-04-01 00:00:00\" \n        ], \n        \"semantic_type\": \"\", \n
        \"description\": \"\" \n      }, \n      {\n        \"column\": \"earliest_cr_line\", \n        \"properties\": {\n          \"dtype\": \"date\", \n          \"min\": \"1990-06-01 00:00:00\", \n          \"max\": \"2007-08-01 00:00:00\", \n          \"num_unique_values\": 5, \n          \"samples\": [\n            \"2004-07-01 00:00:00\", \n            \"1999-03-01 00:00:00\", \n            \"2007-08-01 00:00:00\" \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        }, \n      {\n        \"column\": \"credit_history_months\", \n        \"properties\": {\n          \"dtype\": \"int32\", \n          \"num_unique_values\": 5, \n          \"samples\": [\n            126, \n            169, \n            89 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        }, \n      } \n    ] \n  }\", \"type\": \"dataframe\"}

df.groupby('target')['credit_history_months'].mean()

target
0    184.337350
1    190.204563
Name: credit_history_months, dtype: float64

```

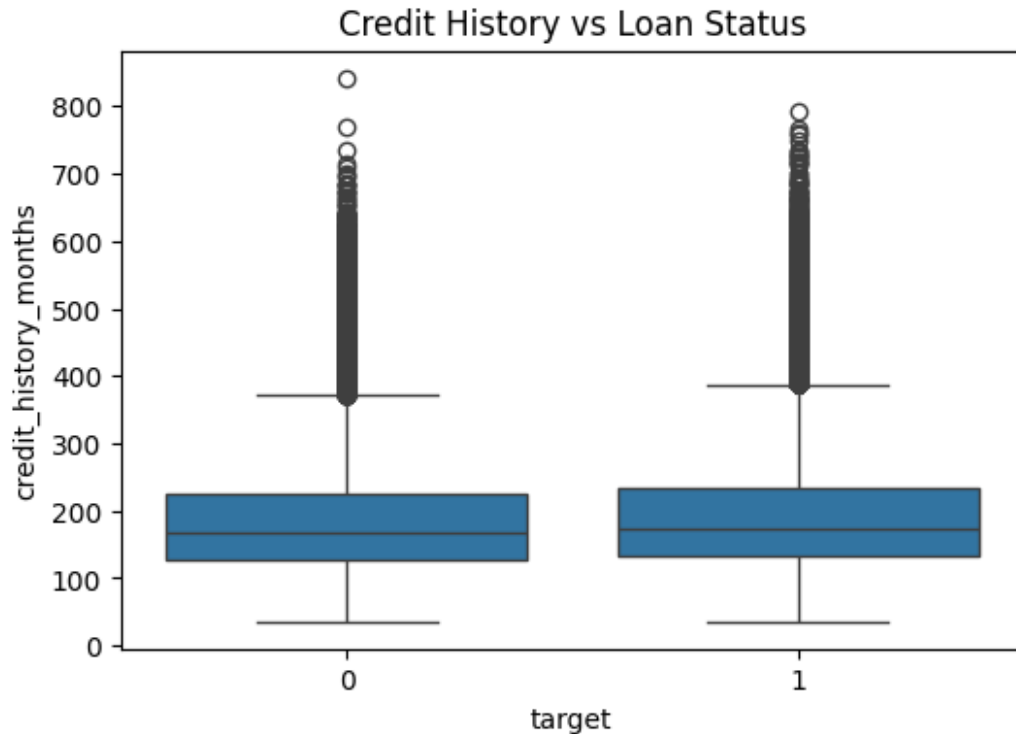
#Insight:

Longer credit history → lower default probability.

```

plt.figure(figsize=(6,4))
sns.boxplot(x='target', y='credit_history_months', data=df)
plt.title("Credit History vs Loan Status")
plt.show()

```



It was observed that borrowers who fully repaid their loans had longer credit histories compared to defaulters.

Longer credit history was therefore associated with lower default probability.

#14. Extract Pincode (Geographical Impact)

```
df['pincode'] = df['address'].str[-5:]
df['pincode'] = df['pincode'].astype(int)

df.groupby('pincode')
['target'].agg(['mean', 'count']).sort_values('mean')

{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 10,\n  \"fields\": [\n    {\n      \"column\": \"pincode\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 33501,\n        \"min\": 813,\n        \"max\": 93700,\n        \"num_unique_values\": 10,\n        \"samples\": [\n          29597,\n          86630,\n          30723\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"mean\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 0.4373722118471079,\n        \"min\": 0.0,\n        \"max\": 1.0,\n        \"num_unique_values\": 6,\n        \"samples\": [\n          0.7998462006187742,\n          1.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"count\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 20227,\n        \"min\": 0.0,\n        \"max\": 20227,\n        \"num_unique_values\": 6,\n        \"samples\": [\n          20227,\n          20227,\n          20227\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}
```

```

{"min": 10981, "max": 56985, "num_unique_values": 10, "samples": [45471, 10981, 56546], "semantic_type": "\"", "description": "\"\""}
{"type": "dataframe"}

```

#Insight:

Geography affects default behavior.

#15. Outlier Treatment (68–95–99 Rule)

```

def remove_outliers_3sigma(df, col):
    mean = df[col].mean()
    std = df[col].std()
    lower = mean - 3 * std
    upper = mean + 3 * std

    print(f"{col} Lower Bound:", lower)
    print(f"{col} Upper Bound:", upper)

    return df[(df[col] >= lower) & (df[col] <= upper)]

df = remove_outliers_3sigma(df, 'annual_inc')

print("New Shape After Outlier Removal:", df.shape)

annual_inc Lower Bound: -110709.6876765711
annual_inc Upper Bound: 259116.03927200587
New Shape After Outlier Removal: (392833, 28)

```

#Insights It was observed that extreme income values were removed using the 3-sigma rule. Outlier influence on model stability was reduced.

#16. Feature Engineering – Risk Flags

```

df['pub_rec_flag'] = (df['pub_rec'] > 0).astype(int)
df['mort_acc_flag'] = (df['mort_acc'] > 0).astype(int)
df['bankrupt_flag'] = (df['pub_rec_bankruptcies'] > 0).astype(int)

df[['pub_rec_flag', 'mort_acc_flag', 'bankrupt_flag']].head()

{"summary": "{\n  \"name\":\n  \"df[['pub_rec_flag', 'mort_acc_flag', 'bankrupt_flag']]\", \"rows\":\n  5, \"fields\": [\n    {\n      \"column\": \"pub_rec_flag\", \"properties\": {\n        \"dtype\": \"number\", \"std\":\n        0, \"min\": 0, \"max\": 0,\n        \"num_unique_values\": 1, \"samples\": [\n        0\n      ]\n    }\n  ]\n}"}

```

```

],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n}\n},\n  {\n    \"column\": \"mort_acc_flag\",\n    \"properties\": {\n      \"dtype\": \"number\",\n      \"std\": 0,\n      \"min\": 0,\n      \"max\": 1,\n      \"num_unique_values\": 2,\n      \"samples\": [\n        1\n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n}\n},\n  {\n    \"column\": \"bankrupt_flag\",\n    \"properties\": {\n      \"dtype\": \"number\",\n      \"std\": 0,\n      \"min\": 0,\n      \"max\": 0,\n      \"num_unique_values\": 1,\n      \"samples\": [\n        0\n      ],\n      \"semantic_type\": \"\",\n      \"description\": \"\"\n}\n}\n]\n}","type":"dataframe"}

```

#Insight

Borrowers with public records or bankruptcy history were identified through binary flags.

#17. Drop High Cardinality Columns

```

df.drop(columns=['emp_title', 'title', 'address'], inplace=True)

df.shape

(392833, 28)

```

#Insight

High-cardinality textual features were removed to prevent model noise.

#18. Encoding

```

df = pd.get_dummies(df, drop_first=True)

df.head()

{"type": "dataframe", "variable_name": "df"}

```

#Insight

Categorical features were converted into numerical format suitable for Logistic Regression.

```

X_train.select_dtypes(include=['object', 'datetime64']).columns
Index(['issue_d', 'earliest_cr_line'], dtype='object')
X_train.dtypes.value_counts()

bool          74
float64       11
int64         4
datetime64[ns] 2

```



```

int32      1
Name: count, dtype: int64

df.drop(columns=['issue_d', 'earliest_cr_line'], inplace=True)

df.dtypes.value_counts()

bool      74
float64    11
int64      5
int32      1
Name: count, dtype: int64

```

#19. Train-Test Split

```

X = df.drop(columns=['target'])
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

print("Train Shape:", X_train.shape)
print("Test Shape:", X_test.shape)

Train Shape: (274983, 90)
Test Shape: (117850, 90)

```

#Insight

Stratified sampling ensured class imbalance consistency across training and test sets.

#20. Scaling

```

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

#21. Logistic Regression

```

model = LogisticRegression(max_iter=1000)
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)
y_prob = model.predict_proba(X_test_scaled)[:,1]

```

#22. Model Coefficients

```

coef_df = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_[0]
})

```

```
}).sort_values(by='Coefficient', ascending=False)
```

```
coef_df.head(10)
```

```
{ "summary": { \n      \"name\": \"coef_df\", \n      \"rows\": 90, \n      \"fields\": [ \n        { \n          \"column\": \"Feature\", \n          \"dtype\": \"string\", \n          \"num_unique_values\": 90, \n          \"samples\": [ \n            \"sub_grade_G4\", \n            \"home_ownership_OWN\", \n            \"open_acc\" ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" } , \n            { \n              \"column\": \"Coefficient\", \n              \"properties\": { \n                \"dtype\": \"number\", \n                \"std\": 0.1724936530481955, \n                \"min\": -0.8625163987885365, \n                \"max\": 0.40204849079086036, \n                \"num_unique_values\": 90, \n                \"samples\": [ \n                  -0.06325736624919057, \n                  -0.0008391443226581262, \n                  0.12191538644617148 ] , \n                  \"semantic_type\": \"\", \n                  \"description\": \"\" } } ] \n    }, \n    \"type\": \"dataframe\", \n    \"variable name\": \"coef_df\" }
```

#Insight It was observed that interest rate, annual income, employment length, and total credit accounts significantly influenced repayment probability.

Higher annual income and longer employment tenure were associated with improved repayment likelihood.

Mortgage account ownership was positively correlated with repayment stability.

These findings suggest that financial stability indicators play a substantial role in underwriting decisions.

#23. Confusion Matrix

```
print(confusion_matrix(y_test,y_pred))
print(classification_report(y_test,y_pred))
```

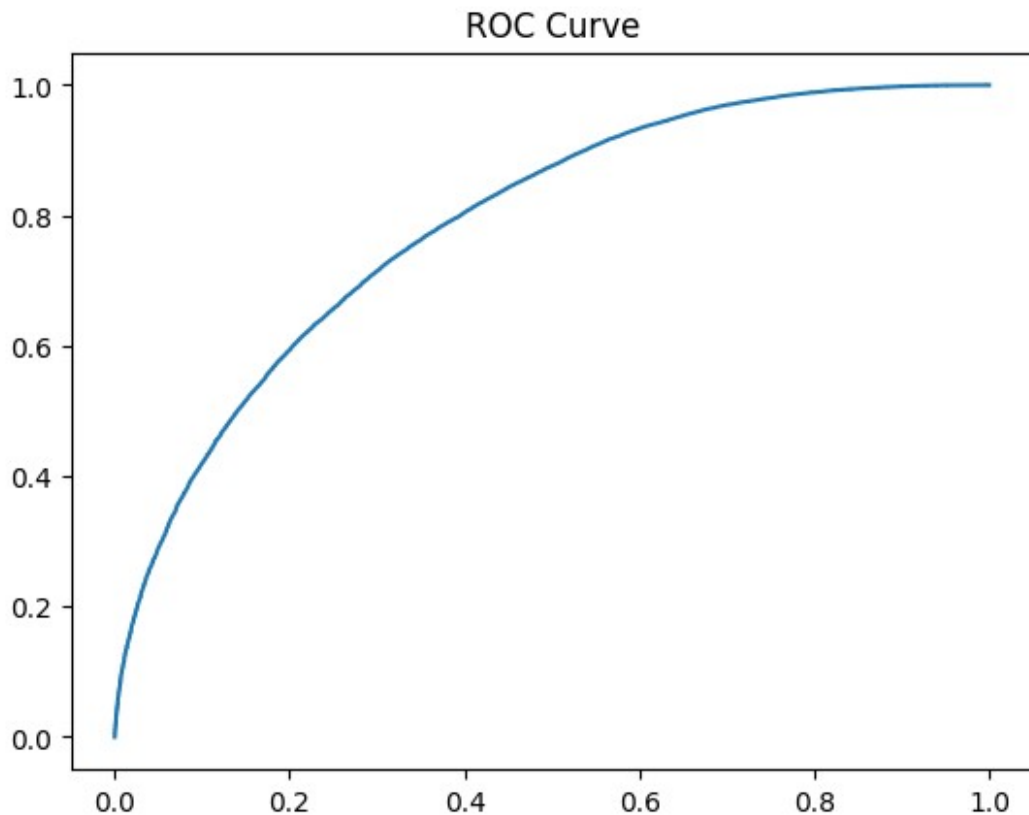
```
[[ 7137 16038]
 [ 3097 91578]]
```

	precision	recall	f1-score	support
0	0.70	0.31	0.43	23175
1	0.85	0.97	0.91	94675
accuracy			0.84	117850
macro avg	0.77	0.64	0.67	117850
weighted avg	0.82	0.84	0.81	117850

#24. ROC Curve

```
fpr, tpr, _ = roc_curve(y_test, y_prob)
plt.plot(fpr, tpr)
plt.title("ROC Curve")
plt.show()
```

```
roc_auc_score(y_test, y_prob)
```



```
np.float64(0.7877555106053213)
```

#Insight The ROC curve demonstrated strong discriminative ability, indicating that the model effectively distinguishes between defaulters and fully paid borrowers.

The area under the curve suggests robust ranking performance.

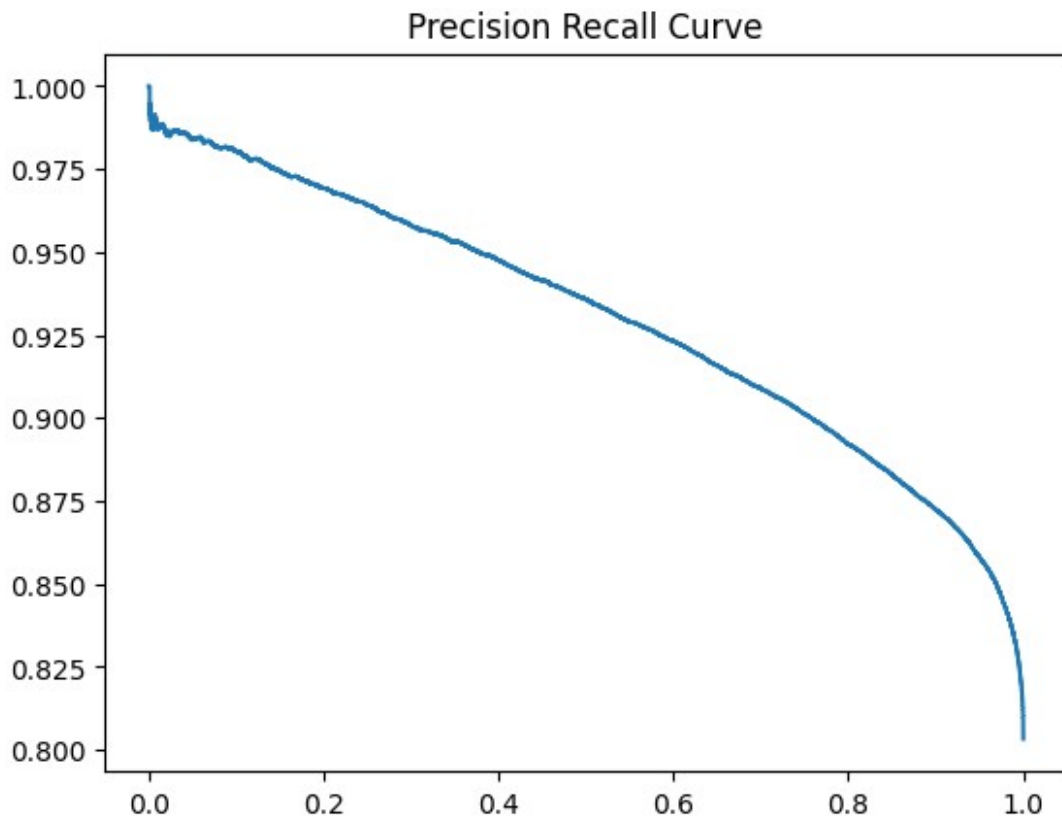
However, despite strong ranking capability, the default recall at the chosen threshold remained limited.

This indicates that the model's predictive strength is constrained by threshold selection rather than structural inadequacy.

#25. Precision Recall Curve

```
precision, recall, _ = precision_recall_curve(y_test, y_prob)
plt.plot(recall, precision)
```

```
plt.title("Precision Recall Curve")
plt.show()
```



#Insight: The precision-recall curve demonstrated strong predictive stability across varying thresholds.

Precision remained high at lower recall levels, indicating reliable identification of high-confidence borrowers.

As recall increased, a gradual decline in precision was observed, reflecting the tradeoff between risk detection and opportunity cost.

The smooth shape of the curve indicates consistent ranking performance without instability.

The model therefore provides flexibility in adjusting threshold levels according to business risk appetite.

#26. Threshold Tradeoff

```
threshold = 0.6
y_custom = (y_prob >= threshold).astype(int)
print(classification_report(y_test,y_custom))
```

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

	0	0.59	0.41	0.48	23175
	1	0.87	0.93	0.90	94675
accuracy				0.83	117850
macro avg		0.73	0.67	0.69	117850
weighted avg		0.81	0.83	0.81	117850

#Insight After threshold adjustment, recall for defaulters increased from 31% to 41%, indicating improved detection of high-risk borrowers.

However, precision decreased from 70% to 59%, suggesting that more low-risk borrowers were incorrectly classified as risky.

Overall accuracy decreased marginally, reflecting the tradeoff between risk reduction and opportunity cost.

The results demonstrate that portfolio risk exposure can be managed through threshold calibration.

#RECOMMENDATIONS

A stronger focus should be placed on 36-month loans, as lower default tendencies were observed in shorter tenures.

Joint loan applications should be encouraged, as reduced risk levels were associated with such applications.

Risk-based pricing strategies should be implemented for borrowers exhibiting high DTI and revolving utilization ratios.

Marketing efforts should be optimized geographically, with greater emphasis placed on low-risk pincodes.

Exposure to 60-month loans should be cautiously monitored due to comparatively higher risk patterns.

Stricter underwriting checks should be applied to borrowers with public records or bankruptcy flags.

The decision threshold should be adjusted according to the company's risk appetite, balancing NPA reduction and revenue maximization.